

Reporte: Actividad 4.1

Alumnos:

- Juan Carlos Malanco Rodriguez
- Isabel Kyra Romero Borunda
- Jocelyn Flores Gutiérrez

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de Aplicaciones Móviles

Fecha: 10/09/2026

TaskMaster: gestor de tareas con Room



Descripción

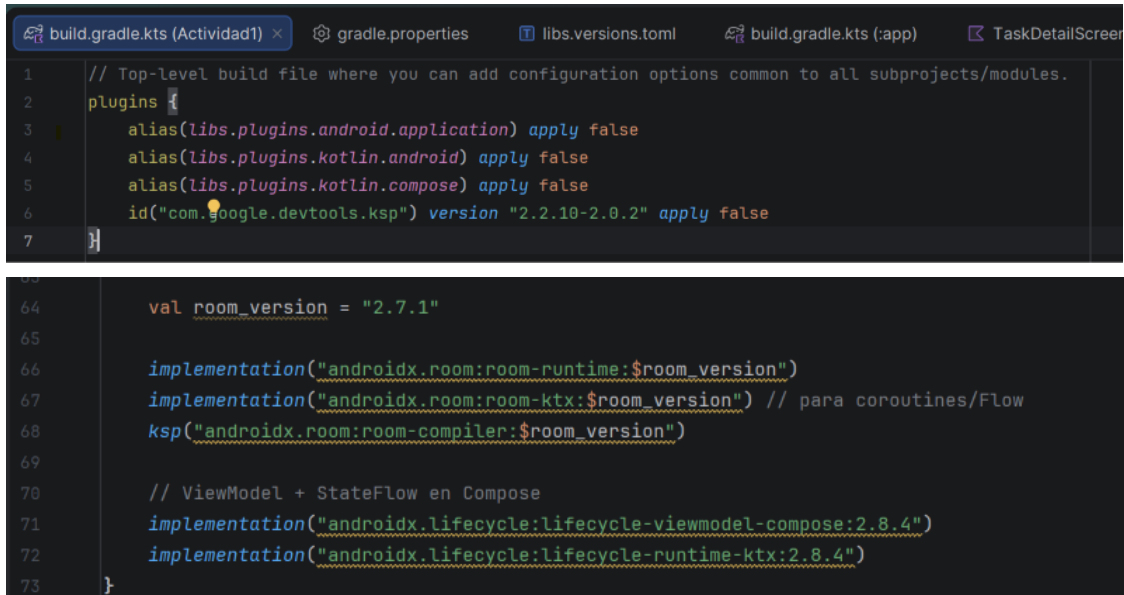
Genera una aplicación de gestión de tareas que utilice *Room* para persistencia de datos locales, integrando operaciones CRUD, validaciones de entrada y una interfaz reactiva con Jetpack Compose.

Instrucciones

1. Configuración inicial:
 - Crea un nuevo proyecto en Android Studio con Jetpack Compose.
 - Agrega las dependencias de Room al archivo build.gradle (usar versión 2.7.1 o superior).
2. Modelado de datos:
 - Define una entidad Task con campos: id (autogenerado), title (String), description (String), isCompleted (Boolean) y dueDate (Long).
 - Crea un DAO con métodos para insertar, eliminar, actualizar y listar tareas (usar Flow para reactividad).
3. Configuración de la base de datos:
 - Implementa una clase AppDatabase abstracta con patrón Singleton.
 - Define un repositorio (TaskRepository) que use el DAO para operaciones.
4. Integración con ViewModel:
 - Crea un TaskViewModel que exponga los datos mediante StateFlow y maneje eventos como addTask o deleteTask.
 - Usa viewModelScope para operaciones asíncronas.
5. Diseño de UI:
 - Implementa una pantalla con:
 - LazyColumn para mostrar tareas.
 - FloatingActionButton para abrir un diálogo de creación.
 - Switch para marcar tareas como completadas.

1. Configuración inicial:

- Crea un nuevo proyecto en Android Studio con Jetpack Compose.
- Agrega las dependencias de Room al archivo build.gradle (usar versión 2.7.1 o superior).



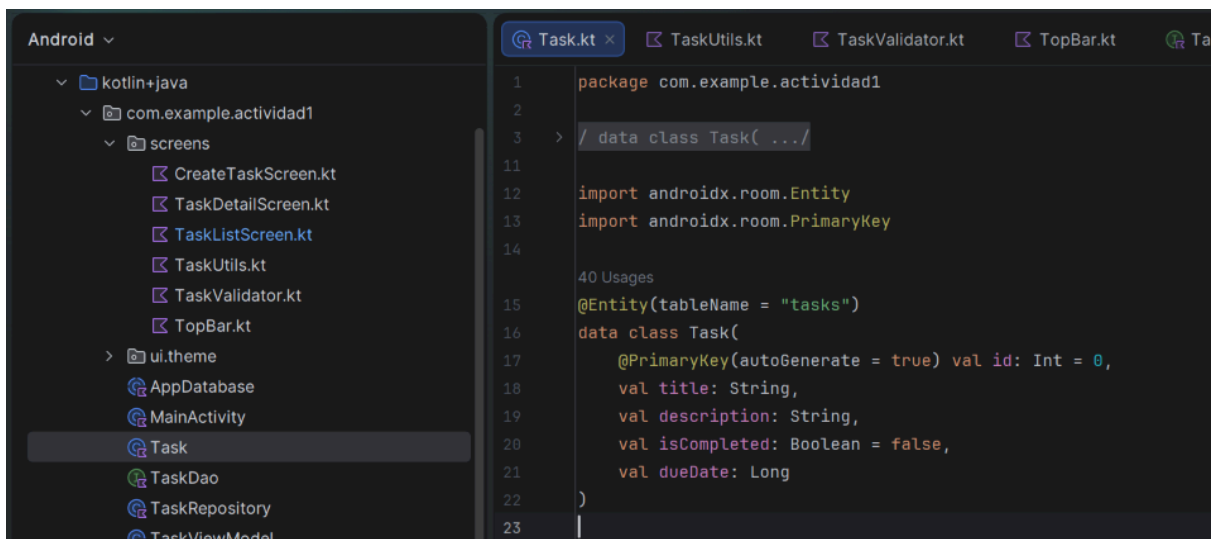
The screenshot shows the Android Studio interface with the 'build.gradle.kts (Actividad1)' file open. The file contains the following Kotlin code:

```
1 // Top-level build file where you can add configuration options common to all subprojects/modules.
2 plugins {
3     alias(libs.plugins.android.application) apply false
4     alias(libs.plugins.kotlin.android) apply false
5     alias(libs.plugins.kotlin.compose) apply false
6     id("com.google.devtools.ksp") version "2.2.10-2.0.2" apply false
7 }
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64 val room_version = "2.7.1"
65
66 implementation("androidx.room:room-runtime:$room_version")
67 implementation("androidx.room:room-ktx:$room_version") // para coroutines/Flow
68 ksp("androidx.room:room-compiler:$room_version")
69
70 // ViewModel + StateFlow en Compose
71 implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.4")
72 implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.4")
73 }
```

Hubo varios problemas para implementar una versión más reciente de ksp, se intentó poner un configuración así pero pedía una coincidencia de versión de ksp con la versión de Kotlin, por lo que se modificó para solucionar ese error y no se usó la versión más reciente. Con room no hubo problema.

2. Modelado de datos:

- Define una entidad Task con campos: id (autogenerado), title (String), description (String), isCompleted (Boolean) y dueDate (Long).
- Crea un DAO con métodos para insertar, eliminar, actualizar y listar tareas (usar Flow para reactividad).

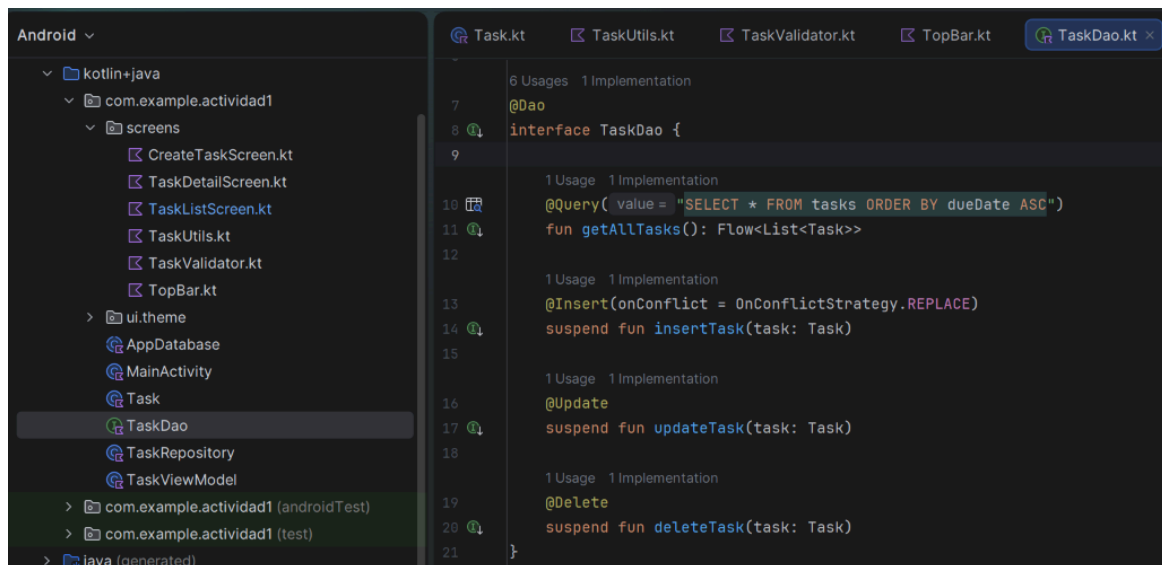


The screenshot shows the Android Studio interface with the 'Task.kt' file open. The file contains the following Kotlin code:

```
1 package com.example.actividad1
2
3 > / data class Task( .../
4
5
6
7
8
9
10
11 import androidx.room.Entity
12 import androidx.room.PrimaryKey
13
14 40 Usages
15 @Entity(tableName = "tasks")
16 data class Task(
17     @PrimaryKey(autoGenerate = true) val id: Int = 0,
18     val title: String,
19     val description: String,
20     val isCompleted: Boolean = false,
21     val dueDate: Long
22 )
23
```

The left sidebar shows the project structure:

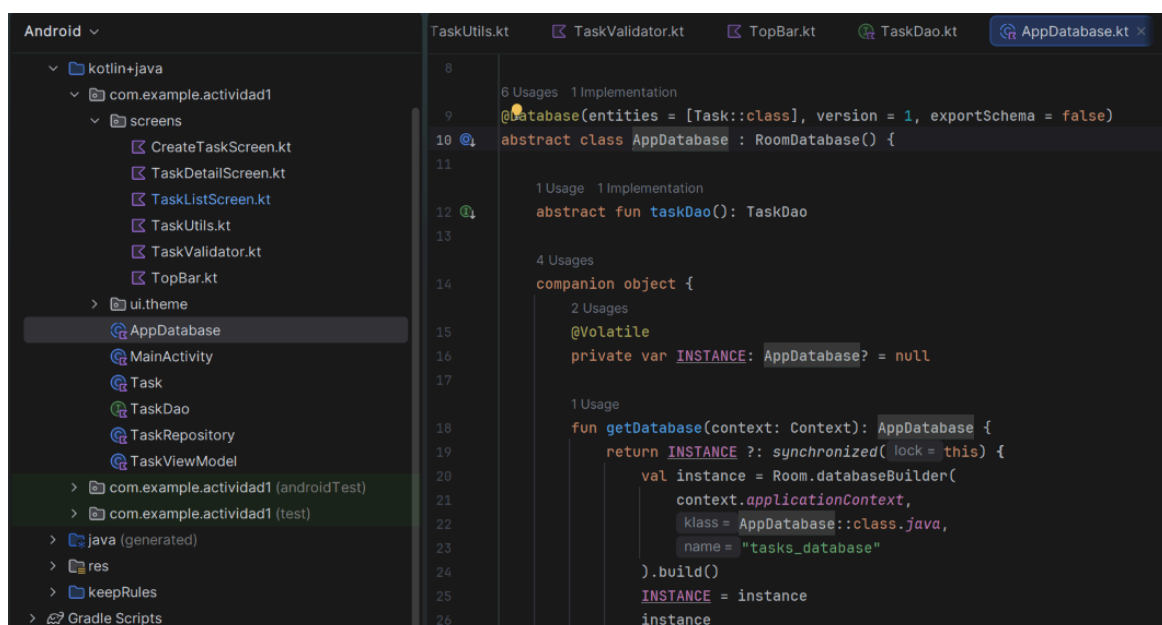
- Android > kotlin+java > com.example.actividad1 > screens
 - CreateTaskScreen.kt
 - TaskDetailScreen.kt
 - TaskListScreen.kt
 - TaskUtils.kt
 - TaskValidator.kt
 - TopBar.kt
- ui.theme
 - AppDatabase
 - MainActivity
 - Task
 - TaskDao
 - TaskRepository
 - TaskViewModel



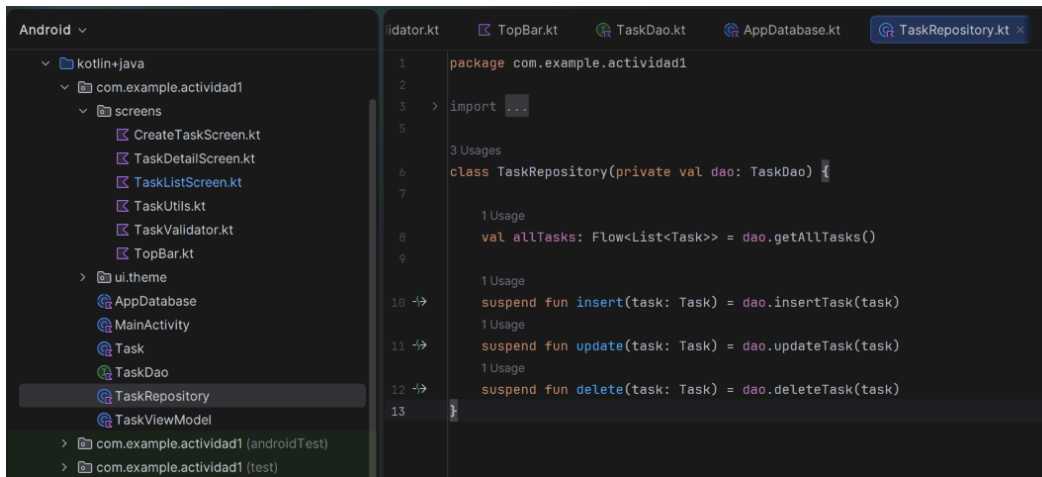
Anteriormente teníamos Task como una clase, no como una entidad, y lo modificamos y agregamos algunas de las propiedades / columnas que nos faltaban: descripción y fecha, y al estatus de completado le cambiamos el nombre para que coincida con el de las instrucciones.

3. Configuración de la base de datos:

- Implementa una clase AppDatabase abstracta con patrón Singleton.
- Define un repositorio (TaskRepository) que use el DAO para operaciones.

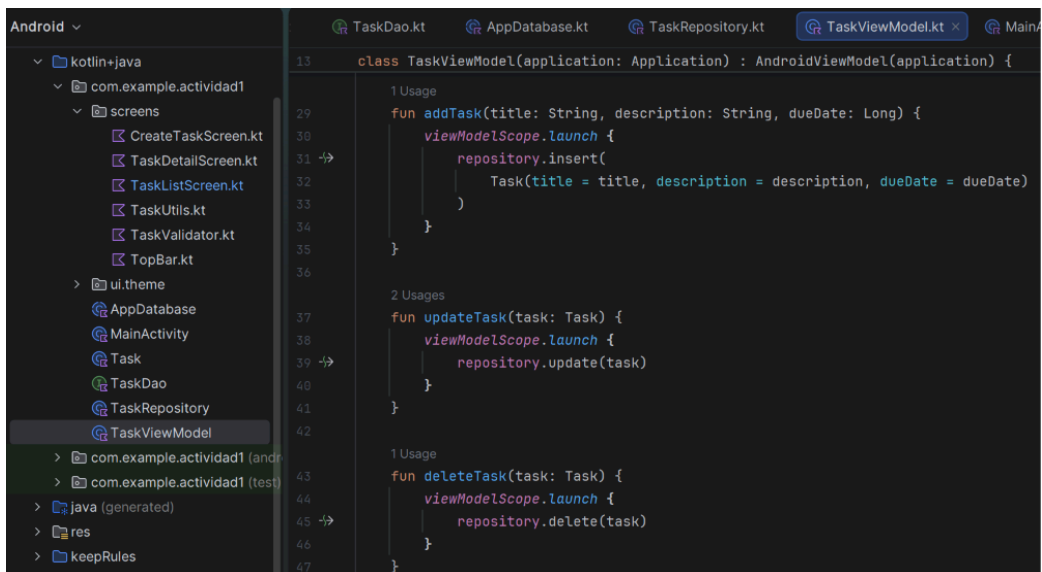
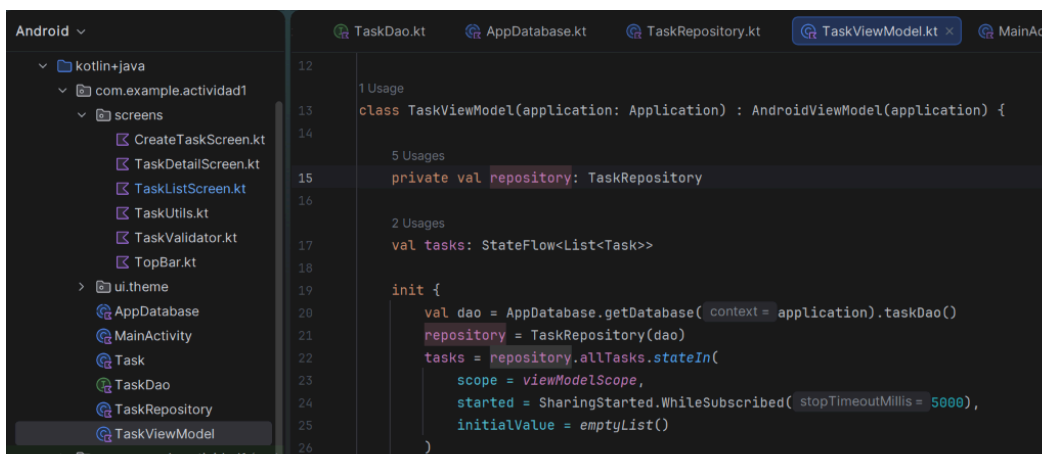


Se utilizaron las operaciones Dao en el repositorio para hacer las acciones CRUD (editar, consultar/mostrar todas las tareas, crear y eliminar).



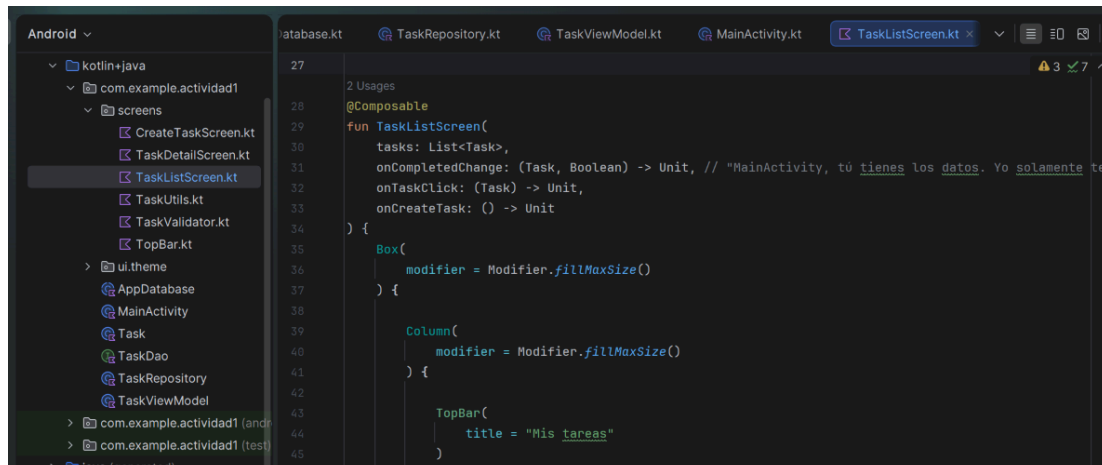
4. Integración con ViewModel:

- Crea un TaskViewModel que exponga los datos mediante StateFlow y maneje eventos como addTask o deleteTask.
- Usa viewModelScope para operaciones asíncronas.



5. Diseño de UI:

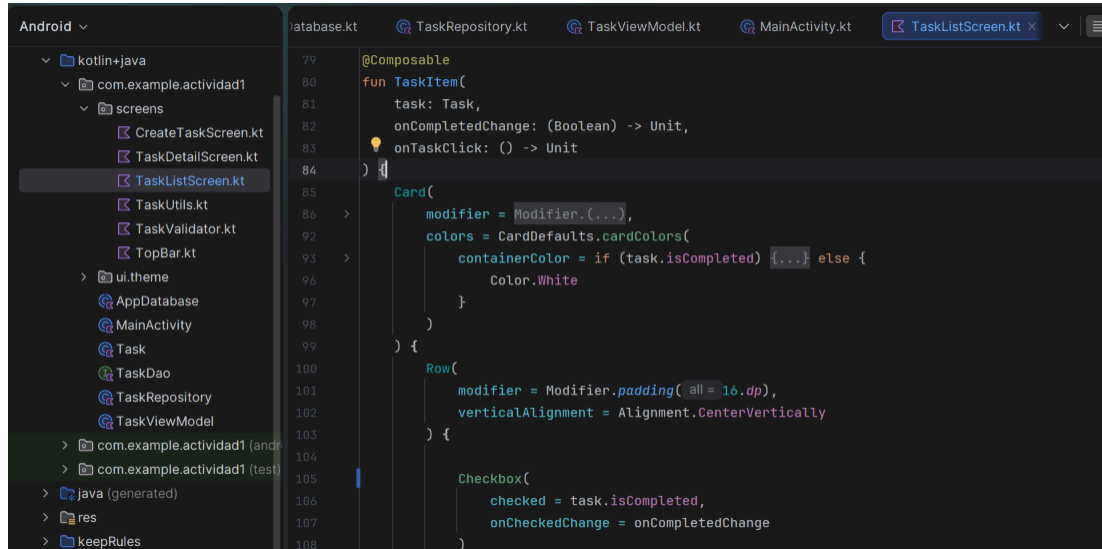
- Implementa una pantalla con:
 - LazyColumn para mostrar tareas.
 - FloatingActionButton para abrir un diálogo de creación.
 - Switch para marcar tareas como completadas.



```
27 2 Usages
28 @Composable
29 fun TaskListScreen(
30     tasks: List<Task>,
31     onCompletedChange: (Task, Boolean) -> Unit, // "MainActivity, tú tienes los datos. Yo solamente te
32     onTaskClick: (Task) -> Unit,
33     onCreateTask: () -> Unit
34 ) {
35     Box(
36         modifier = Modifier.fillMaxSize()
37     ) {
38         Column(
39             modifier = Modifier.fillMaxSize()
40         ) {
41             TopBar(
42                 title = "Mis tareas"
43             )
44         }
45     }
```



```
46
47     LazyColumn(
48         modifier = Modifier(...)
49     ) {
50         items(tasks) { task ->
51             TaskItem(...)
52         }
53     }
54
55     FloatingActionButton(...) {
56         Text("+")
57     }
58 }
```



```
79 @Composable
80 fun TaskItem(
81     task: Task,
82     onCompletedChange: (Boolean) -> Unit,
83     onTaskClick: () -> Unit
84 ) {
85     Card(
86         modifier = Modifier(...),
87         colors = CardDefaults.cardColors(
88             containerColor = if (task.isCompleted) {...} else {
89                 Color.White
90             }
91         )
92     ) {
93         Row(
94             modifier = Modifier.padding(all = 16.dp),
95             verticalAlignment = Alignment.CenterVertically
96         ) {
97             Checkbox(
98                 checked = task.isCompleted,
99                 onCheckedChange = onCompletedChange
100             )
101         }
102     }
103 }
```

LazyColumn va agregando las tareas que se van creando y se eligió cambiar el switch por el checkbox porque la experiencia era muy extraña para una lista de tareas.

El FloatingActionButton más que abrir una pequeña ventana, llama a una vista nueva para crear la tarea.

Capturas

